NAVIGATION FIX REPORT · VOL. 2

Command Palette
Navigation Fix Guide

Fixing CommandResult misuse: back navigation, post-command flow, and stuck-query behaviour

ISSUE REPORTED

After running one command, can't run another — palette doesn't go back to main menu.

EXTENSION	ROOT CAUSE	REPORT DATE	VOLUME
DiskAnalyzerExtension 1.2.0.0	CommandResult.KeepOpen() wrong return value	June 14, 2026	Vol. 2 of 2

SECTION 1

Root Cause — KeepOpen Is the Scaffolded Default

When the extension template generates a list item, it wires each item to `NoOpCommand`, which always returns `CommandResult.KeepOpen()`. The official SDK documentation states:

> → **Official SDK — KeepOpen behaviour**
> KeepOpen does nothing. It leaves the palette in its current state, with the current page stack and current query text unchanged.

This causes three compounding problems after a command runs:

#	Symptom	Caused By
1	Typed query stays in search box	Page stack not unwound — query is per-page
2	Only filtered results visible	Query text filters the list; KeepOpen keeps it
3	Can't invoke another command	Palette is 'stuck' on current page/state

All 7 CommandResult Types — Reference Table

Choose the return value based on what should happen *immediately* after the command action completes:

CommandResult	Palette Behaviour After Invoke	Use When	Rec.
`Dismiss()`	Closes palette; resets to home on next open	One-off actions: open file, launch tool, copy text	★★★
`ShowToast(msg)`	Shows desktop toast, then auto-dismisses	Any fire-and-forget action that should confirm completion	★★★
`GoBack()`	Pops one page off the stack; palette stays open	Form submission → back to parent list	★★■
`GoHome()`	Jumps to main palette home; palette stays open	Multi-step flows; after refreshing top-level data	★★■
`GoToPage(args)`	Pushes a new page onto the stack	Drill-down: pick item → open detail page	★★■
`Hide()`	Hides palette; current page preserved for later	Temporarily leaving palette then coming back	★■■
`KeepOpen()`	Does nothing — page/query unchanged	Changing filter/selection with no action taken yet	■■ Rarely

> ✓ **Golden Rule from the SDK Docs**
> If you don't know what else to use, Dismiss() should be your default. It closes the palette cleanly and resets state for the next open.

SECTION 3

Fix 1 — Replace NoOpCommand with InvokableCommand

The scaffolded list items use `NoOpCommand` as a placeholder. It must be replaced with a class that extends `InvokableCommand` and returns the correct `CommandResult`.

Before — Broken Scaffolded Pattern

```
// ■ BROKEN — NoOpCommand always returns KeepOpen
return new ListItem(new NoOpCommand())
{
    Title = "Analyze C:\\"
};
```

After — Correct InvokableCommand

```
// ■ CORRECT — dedicated command with explicit CommandResult
public sealed class OpenDriveCommand : InvokableCommand
{
    private readonly DriveInfo _drive;

    public OpenDriveCommand(DriveInfo drive)
    {
        _drive = drive;
        Name   = $"Open {drive.Name}";
        Icon   = new IconInfo("\uE838");
    }

    public override CommandResult Invoke()
    {
        try
        {
            Process.Start("explorer.exe", _drive.RootDirectory.FullName);
            return CommandResult.ShowToast($"Opened {_drive.Name}");
            // ShowToast auto-Dismisses — palette closes cleanly ■
        }
        catch (Exception ex)
        {
            return CommandResult.ShowToast($"■ {ex.Message}");
        }
    }
}
```

Wiring It Into List Items

```
return _cachedDrives.Select(d =>
{
    long   used = d.TotalSize - d.AvailableFreeSpace;
    double pct  = (double)used / d.TotalSize * 100;

    // ■ Each item gets its own real InvokableCommand
    return (IListItem)new ListItem(new OpenDriveCommand(d))
    {
        Title    = $"{d.Name}  ({pct:F1}% used)",
        Subtitle = $"{FormatBytes(used)} of {FormatBytes(d.TotalSize)}"
    };
}).ToArray();
```

Fix 2 — Navigation by Scenario

Here are all four common navigation patterns with ready-to-use code for each:

Scenario A — One-Off Action (open, run, copy, launch)

```
// User does the thing and is done — close & reset
public override CommandResult Invoke()
{
    OpenDiskReport(_drivePath);
    return CommandResult.Dismiss();       // ■ closes palette
}
```

Scenario B — Action With Confirmation

```
// Same as A, but shows a toast before closing
public override CommandResult Invoke()
{
    var summary = RunQuickScan(_drivePath);
    return CommandResult.ShowToast($"■ {summary}");
    // Dismiss is built into ShowToast — palette closes ■
}
```

Scenario C — Back to Main Menu (palette stays open)

```
// User picks an item; palette stays open at home for next action
public override CommandResult Invoke()
{
    PinDrive(_drivePath);
    return CommandResult.GoHome();        // ■ back to main menu
}
```

Scenario D — Back One Page (form → parent list)

```
// Form 'Save' button — goes back to the list that opened the form
public override CommandResult Invoke()
{
    SaveSettings(_formData);
    return CommandResult.GoBack();        // ■ pops page stack
}
```

Scenario E — Navigate to a Sub-Page (drill-down)

```
// Picking a drive opens a detail page for that drive
public override CommandResult Invoke()
{
    var detail = new DriveDetailPage(_drivePath);
    return CommandResult.GoToPage(        // ■ pushes new page
        new GoToPageArgs(detail));
}
```

SECTION 5

Fix 3 — Complete Corrected DiskAnalyzerPage

Here is the full updated page with all three fixes applied: async loading (Vol. 1), try/catch (Vol. 1), and correct `CommandResult` navigation (this guide).

```
// ■■ InvokableCommand ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public sealed class OpenDriveCommand : InvokableCommand
{
    private readonly DriveInfo _drive;
    public OpenDriveCommand(DriveInfo d) {
        _drive=d; Name=$"Open {d.Name}"; Icon=new IconInfo("\uE838");
    }
    public override CommandResult Invoke()
    {
        try {
            Process.Start("explorer.exe", _drive.RootDirectory.FullName);
            return CommandResult.ShowToast($"Opened {_drive.Name}");
        }
        catch (Exception ex) {
            return CommandResult.ShowToast($"■ {ex.Message}");
        }
    }
}

// ■■ ListPage ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
public sealed class DiskAnalyzerPage : ListPage
{
    private DriveInfo[]? _cachedDrives;

    // ■ Fast constructor — no I/O (Vol. 1 Fix 1)
    public DiskAnalyzerPage() {
        Name="Disk Analyzer"; Icon=new IconInfo("\uE9F9");
    }

    // ■ Async placeholder pattern (Vol. 1 Fix 2)
    public override IListItem[] GetItems()
    {
        try {
            if (_cachedDrives is null) {
                _ = LoadDrivesAsync();
                return [new ListItem(new NoOpCommand()) {
                    Title="Scanning...", Subtitle="Please wait" }];
            }
            // ■ Real InvokableCommand on each item (Vol. 2 Fix 1)
```

SECTION 6

Quick Decision Guide — Which CommandResult to Use

After my command runs, should the palette...

→ **Close completely and reset to home?**

```
CommandResult.Dismiss()
```

→ **Close AND show a confirmation message?**

```
CommandResult.ShowToast("message")
```

→ **Stay open at the palette home screen?**

```
CommandResult.GoHome()
```

→ **Go back one page (e.g. form → list)?**

```
CommandResult.GoBack()
```

→ **Open a specific sub-page (drill-down)?**

```
CommandResult.GoToPage(new GoToPageArgs(page))
```

→ **Do nothing / stay on this exact page?**

```
CommandResult.KeepOpen()  ← rarely the right choice
```

★ **If unsure — always start with Dismiss()**
Dismiss() is the safest default. It closes the palette cleanly, resets all state, and gives users a clean slate for the next open. Upgrade to ShowToast when you want to confirm the action.

> ! **Never Use KeepOpen on Action Commands**
> KeepOpen is only correct when the command itself IS the filtering/selection action and no real work has been performed yet. Using it after an action executes causes exactly the stuck-navigation bug you experienced.

SECTION 7

Fix Checklist

- ■ Find every new ListItem(new NoOpCommand()) in your page file(s)
- ■ Replace each NoOpCommand with a dedicated InvokableCommand subclass
- ■ In each Invoke(), choose the correct CommandResult (see Section 6)
- ■ Default to CommandResult.Dismiss() for any one-off action
- ■ Use CommandResult.ShowToast(msg) when you want a confirmation message
- ■ Use CommandResult.GoBack() when a form Save/Submit should return to parent list
- ■ Use CommandResult.GoHome() for multi-step flows that stay in palette
- ■ Wrap the Invoke() body in try/catch — return ShowToast on error
- ■ Redeploy: Right-click project → Deploy
- ■ Reload: Open CmdPal → type 'reload' → Reload Command Palette extensions
- ■ Test: invoke a command, verify palette navigates correctly, invoke a second command